BCSE307P Compiler Design Lab

Innovative Project

FRACTL

A compiler from L-system grammars to native code via LLVM

by

Reg No	Team Member Names

Under the Supervision of

Prof. NIVITHA K

Assistant Professor Sr. Grade-2

School of Computer Science and Engineering



November 2026

1. Abstract

FRACTL is a small domain-specific language and compiler for Lindenmayer systems (L-systems). It turns a short text description of a fractal or plant into a drawing, using the classic compiler phases and LLVM as the back end.

A `.fr` source file states an axiom, rewriting rules (optionally random, with weights and a seed) and turtle-graphics parameters. The compiler scans, parses and checks it, expands the grammar, lowers it to a turtle-command IR, and generates LLVM IR. The IR is then compiled to a native object file or run in memory by LLVM's ORC JIT. A small C runtime executes the turtle commands and writes an SVG (2D) or a 3D segment stream.

The project was built in three stages. Stages 1 and 2 cover 2D fractals (Koch, Dragon, Plant, Tree, Carpet). Stage 3 adds stochastic grammars, recursive code generation, a 3D turtle, multi-fractal scenes (for example a tree among 1,300 independently grown grass tufts) and FRACTL Studio, a browser app that shows every compiler phase and grows the result live.

2. Introduction

Fractals such as the Koch snowflake or a branching plant are generated by repeating one simple rule at many scales. An L-system captures this: start with an axiom string, then rewrite every symbol in parallel using production rules. Interpreting the final string as turtle-graphics commands (`F` forward, `+` and `-` turn, `[` and `]` save and restore the position) draws the shape.

For example, Koch uses the axiom `F--F--F` and the single rule `F -> "F+F--F+F"`. After four passes the string has 1,792 symbols, and 768 of them are forward moves.

Writing such systems directly in a general-purpose language mixes the grammar with drawing code. FRACTL separates them: the user writes only the grammar and the drawing parameters in a purpose-built language, and a real compiler does the rest. The project is a compact, complete example of the compiler phases:

- lexical analysis and LL(1) parsing of a small DSL;
- semantic checks (duplicate fields, bracket balance, weights, ranges);
- grammar expansion and lowering to a turtle intermediate representation;
- optimization, LLVM IR generation, and native or JIT execution against a runtime library.

The front end is pure C++17 with no LLVM dependency. The back end uses LLVM 20 and a C runtime. The two sides meet at two frozen interfaces, the AST and the turtle IR.

3. Novelty in Project

The main novelty is that FRACTL compiles the grammar instead of interpreting it, so the generated code stays the size of the grammar however large the picture grows.

- **Recursive code generation.** The compiler emits one LLVM function per rule symbol, taking the number of passes left. For `meadow.fr` (1,301 instances) the IR is 531 lines instead of 201,463 when unrolled, and the object file is 91 KB instead of 11.9 MB.
- **Stochastic grammars with reproducible output.** Rules take weighted alternatives such as `F -> "F+F" [0.6] | "F-F" [0.4];` plus a `seed:`. A counter-based random generator is implemented

identically in the compiler and in the C runtime, so recursive and unrolled code make exactly the same choices.

- **LLVM ORC JIT.** The program is compiled to machine code in memory and run directly, with no object file or linker.
- **A 3D turtle.** Pitch, roll, tubes, flat blades for grass and leaves, tapering width and colour palettes.
- **Scenes.** One file can hold several fractals and a `scene` that places or scatters them. Each scattered copy gets its own seed, so every tuft grows differently from one grammar.
- **Peephole optimizer.** `--opt` merges adjacent constant rotations and never merges rotations about different axes, because 3D rotations do not commute.
- **FRACTL Studio.** A browser app with a hand-written WebGL2 renderer. It shows tokens, AST, expanded string, turtle IR, LLVM IR and per-phase timings, and replays growth in the order the compiled program drew it.
- **Backward compatibility.** The six Stage 2 drawings are byte-identical in Stage 3, and a test enforces it.

4. Proposed Work

FRACTL is organised as a front end (phases 1 to 4) and a back end (phases 5 to 7) joined by two frozen interfaces, `ast.hpp` and `ir.hpp`.

4.1 The language

```
fractal Koch {
    axiom: "F--F--F";
    rules { F -> "F+F--F+F"; }
    angle: 60;  iterations: 4;  step: 4;
    start: (150, 350);  heading: 0;
}
```

The grammar is LL(1) because every field starts with a distinct keyword. The fields are `axiom`, `rules`, `angle`, `iterations`, `step`, `start` and `heading`, and in Stage 3 also `seed`, `dimensions`, `width`, `taper` and `colors`. Comments (`//`, `#`, `/ /`) are skipped.

```
program     -> fractalDecl EOF
fractalDecl -> 'fractal' IDENT '{' field* '}'
field       -> 'axiom' ':' STRING ';' | 'angle' ':' NUMBER ';' | ... | 'rules' '{' ruleDecl* '}'
ruleDecl    -> SYMBOL '->' STRING [ '[' NUMBER ']' ] { '|' STRING [ '[' NUMBER ']' ] } ';'
```

4.2 Compiler phases

#	Phase	What FRACTL does	Files
1	Lexical analysis	Hand-written scanner with one character of lookahead. Produces keyword, identifier, string, number and punctuation tokens with line numbers. Throws `LexError`.	`lexer.hpp`
2	Syntax analysis	Recursive-descent LL(1) parser building the AST (`FractalDecl`, `Production`, `RuleSet`, and `SceneDecl` in Stage 3). Throws `ParseError: line N: expected ...`.	`parser.hpp`, `ast.hpp`
3	Semantic checks	Duplicate fields and rules, one-character rule symbols, balanced `[ ]`, weights above 0, `dimensions` 2 or 3, `taper` in (0, 1], colour format, unknown fractal in a scene, `iterations` at most 60.	`parser.hpp`
4	Expansion and lowering	Parallel (D0L) rewriting with identity fallback and a blow-up guard, then one turtle command per symbol (`Forward`, `Move`, `TurnLeft`, `Push`, ...). Scene placements are computed here from seeds.	`rewriter.hpp`, `rng.hpp`, `lower.hpp`, `scene.hpp`, `ir.hpp`
5	Optimization	`--opt` folds adjacent constant rotations and deletes runs that add up to a full turn.	`codegen.cpp`
6	Code generation	LLVM IR with one function per rule symbol (recursive mode) or one call per command (`--unroll`). Scenes become a constant `@instances` table and a `fractl_main` loop.	`codegen.cpp`
7	Execution and runtime	Native object plus link, or ORC JIT in memory. `runtime.c` holds the 2D and 3D turtle and writes `.svg` or `.f3d`.	`jit.cpp`, `emit_object.cpp`, `runtime.c`

4.3 Turtle alphabet

Symbol	Meaning
F	Forward, drawing a tube of the current width (a line in 2D)
G	Forward, drawing a flat blade (grass, leaves)
f	Move without drawing
+ -	Turn left or right by `angle`
& ^	Pitch down or up (3D)
\ /	Roll left or right (3D)
|	Turn around 180 degrees
!	Multiply the width by `taper`
'	Switch to the next colour in `colors`
[ ]	Push or pop the whole turtle state
any other letter (X, Y, ...)	Structural, draws nothing

In a 2D fractal the 3D-only symbols are ignored, so older grammars are unaffected.

4.4 Error handling and debugging

Every phase reports errors as `line N: message`. The front-end driver `fr_frontend` can print the output of any phase (`--tokens`, `--ast`, `--expand`, `--ir`, `--stats`). The expanded string is the most useful view: a wrong string points to the rewriter or the grammar, and a right string with a wrong picture points to lowering or the back end.

5. Implementation & Screenshots

The front end is header-only C++17, built with MinGW g++ and on Linux with `-Wall -Wextra` and no warnings. The back end is C++ against LLVM 20 plus a C runtime. Build and run with `make` and `make test`, or with `build.ps1` on Windows.

5.1 Worked example: koch.fr

1 **Lexer:** produces `fractal`, `Koch`, `{`, `axiom`, `:`, `"F--F--F"`, `;` and so on (40 tokens).

2 **Parser:** builds a `FractalDecl` with axiom `F--F--F`, rule `F -> F+F--F+F`, angle 60, 4 iterations, step 4, start (150, 350).

3 **Rewriter:** gives `F+F--F+F--F+F--F+F--F+F--F+F` after one pass and 1,792 symbols after four.

4 **Lowering:** gives 1,792 turtle commands, 768 of them `Forward 4` (3 x 4^4).

5 **Code generation:** emits LLVM IR calling `turtle_forward` and `turtle_turn`, then an object file or the JIT.

6 **Runtime:** executes the calls and writes `koch.svg`.

5.2 Results

Demo	Iterations	Expanded symbols	Turtle commands	Forward	Push = Pop
`triangle.fr`	0	7	7	3	0
`koch.fr`	4	1,792	1,792	768	0
`dragon.fr`	12	16,382	12,286	4,096	0
`plant.fr`	6	25,159	21,063	6,048	4,095
`tree.fr`	7	13,956	11,769	4,118	2,186
`carpet.fr`	4	9,579	9,579	4,096	0

Recursive code generation compared with unrolled code (Stage 3):

Program	Recursive IR (lines)	Unrolled IR (lines)	Recursive object	Unrolled object
`plant.fr`	70	21,086	2.7 KB	1.0 MB
`dragon.fr`	60	12,309	2.5 KB	751 KB
`meadow.fr` (1,301 instances)	531	201,463	91 KB	11.9 MB

The meadow scene (a tree and 1,300 grass tufts: 1,301 instances and 39,588 segments, of which 1,488 are tubes and 38,100 are blades) builds through the studio in a 102 ms round trip. That is lex, parse and check 0.3 ms, LLVM IR generation 3.0 ms (8 functions, 444 instructions), ORC JIT 4.8 ms, run 51 ms and GPU upload 66 ms (Figure 5).

5.3 Testing

Suite	Result
Frontend tests (`test_frontend.cpp`)	79,580 checks, 0 failed
Runtime tests (`test_runtime.c`)	36 checks, 0 failed
Pipeline tests	8 checks, 0 failed
Mode-equivalence tests (recursive = unrolled, object = JIT, `--opt` = no `--opt`)	59 checks, 0 failed
Golden files (SVG or `.f3d`, plus IR)	16 of 16 match

5.4 FRACTL Studio

Studio (`Launch FRACTL Studio.bat`) is a local web app. **Build** compiles the editor text with `fractlc --jit` and draws the result. A pipeline panel lists each phase with its measured time. The **Stages** tab shows tokens, AST, expanded string, turtle IR and counts, and a drawer shows the generated LLVM IR. Growth replay animates the segments in emission order, and **Export** saves `.svg` or `.obj`.

5.5 Screenshots

Every Studio screenshot below is a live build on the development laptop (Intel integrated graphics). The studio wrote the editor text to a `.fr` file, compiled it with `fractlc --jit`, ran the compiled code and drew its output. Nothing is pre-rendered. The left panel shows the source and the measured time of every compiler phase. The chips at the top right show the counts the compiled program produced.

The 3D models come first, building up from a single grass tuft to the full meadow. All the 2D fractals follow, then screenshots of the compiler's intermediate stages and the Stage 1 toolchain.

5.5.1 3D models in FRACTL Studio

Grass tuft

One clump of grass from a stochastic grammar. `T` picks one of three blade layouts and each `B` picks one of five blade shapes by weight. `G` draws a flat blade, `&` pitches it over and `!` thins it toward the tip.



Figure 1. Grass tuft (`tuft.fr`): 1 instance, 19 blades, 5 LLVM functions, 301 IR instructions; round trip 60 ms

Grass field

The same `Tuft` grammar, placed 1,350 times by a `scene` with two `scatter` statements: 1,000 tufts in 36 clumps and 350 filling the gaps. Each tuft gets its own seed, so no two are alike. The IR is the same size as for one tuft, 5 functions and 301 instructions, because only the run grows.

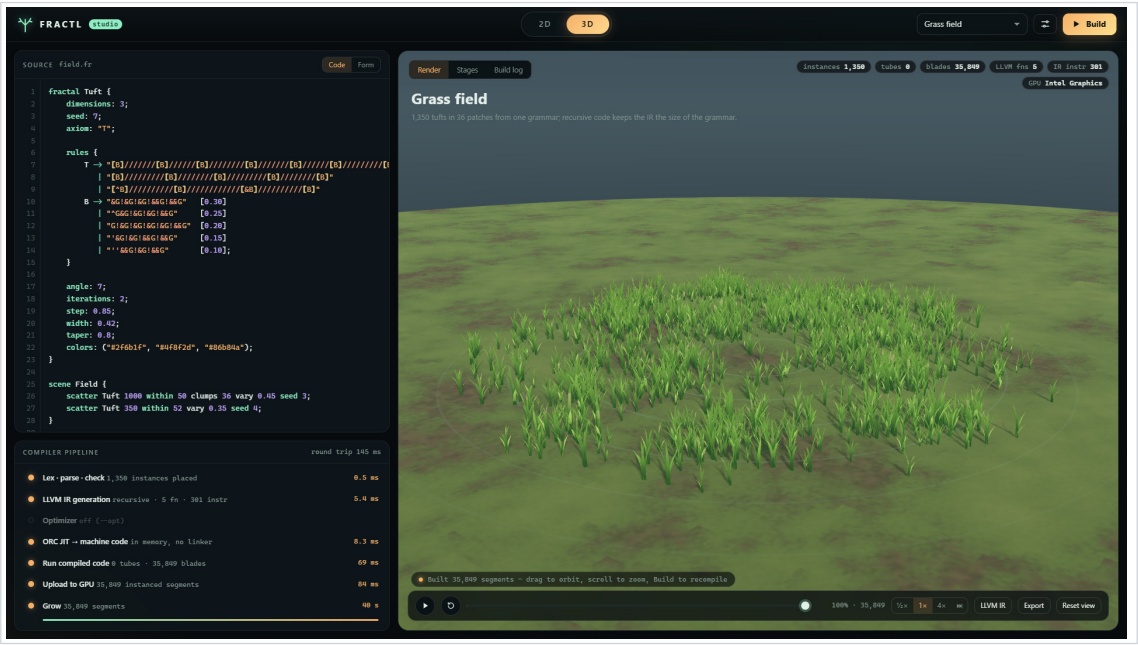


Figure 2. Grass field (`field.fr`): 1,350 instances, 35,849 blades, 5 LLVM functions, 301 IR instructions; JIT 8.3 ms, run 69 ms, round trip 145 ms

Shrub

A three-way branching bush. `A -> [&FA]/////[&FA]/////[&FA]` (or a two-way variant, weight 0.3) pitches each branch out with & and rolls between branches with `/`. `F -> F | FF` gives random internode lengths. `F` draws tubes, whose width tapers with depth.



Figure 3. Shrub (`bush3d.fr`): 1 instance, 913 tubes, 5 LLVM functions, 104 IR instructions; round trip 60 ms

Tree

A stochastic tree with leaves. The wood is drawn with `F` (tubes, thinned by `!` at every fork), and every node grows leaf blades with `G`. The `'` symbol steps through the colour palette, from bark to dark leaf to light leaf. `seed: 42` makes the tree identical on every build.



Figure 4. Tree (`tree3d.fr`): 1 instance, 1,488 tubes, 3,358 blades, 5 LLVM functions, 169 IR instructions; round trip 21 ms

Meadow: the tree in a grass field

Two grammars in one file. The scene places the `Tree` at the origin and scatters 1,300 `Tuft` instances around it in patches, keeping a clear ring around the trunk. The compiled `fractl_main` loops over a constant instance table and calls each fractal's axiom function.



Figure 5. Meadow (`meadow.fr`): 1,301 instances, 1,488 tubes, 38,100 blades, 8 LLVM functions, 444 IR instructions; round trip 102 ms

5.5.2 2D fractals in FRACTL Studio

In 2D the compiled program writes an SVG. The studio colours each stroke by drawing order, from teal (first) to amber (last), so the picture also shows the order in which the turtle moved.

Wildflower (stochastic)

Stage 3's weighted rules in 2D: `F` becomes `F[+F]F[-F]F`, `F[+F]F` or `F[-F]F` with weights 0.34, 0.33 and 0.33, and `seed: 2026` fixes the plant. Changing the seed grows a different plant from the same grammar.

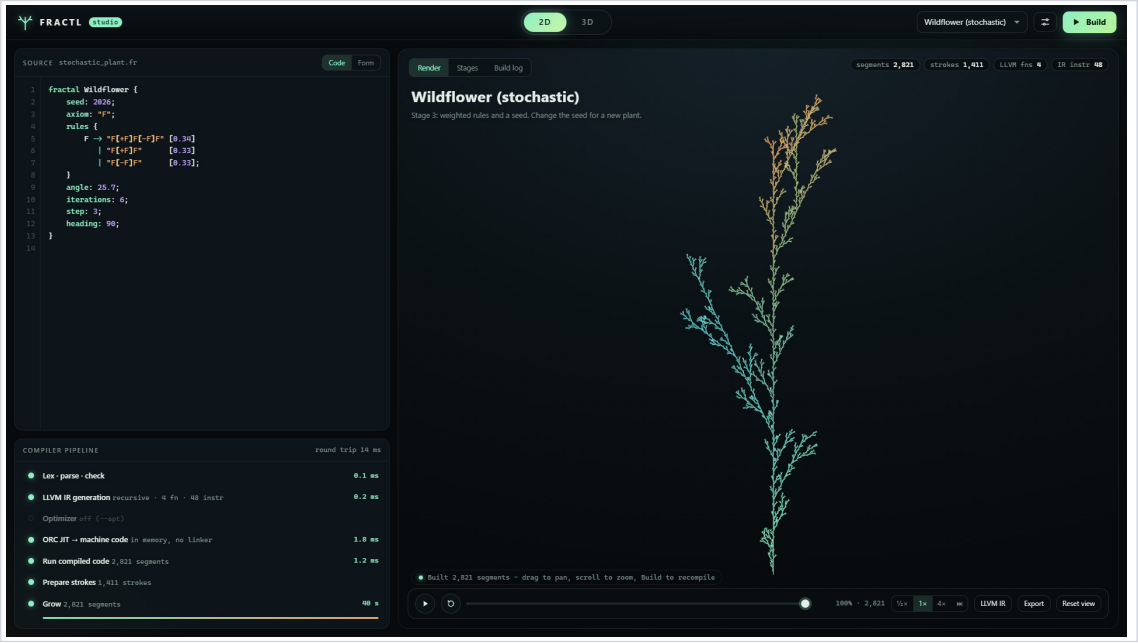


Figure 6. Wildflower (`stochastic_plant.fr`): 2,821 segments, 4 LLVM functions, 48 IR instructions

Fractal plant

`X -> F+[[X]-X]-F[-FX]+X` and `F -> FF`, 25 degree turns, 6 iterations. `[` and `]` save and restore the turtle, which makes the branches. `X` draws nothing; it only steers the growth.

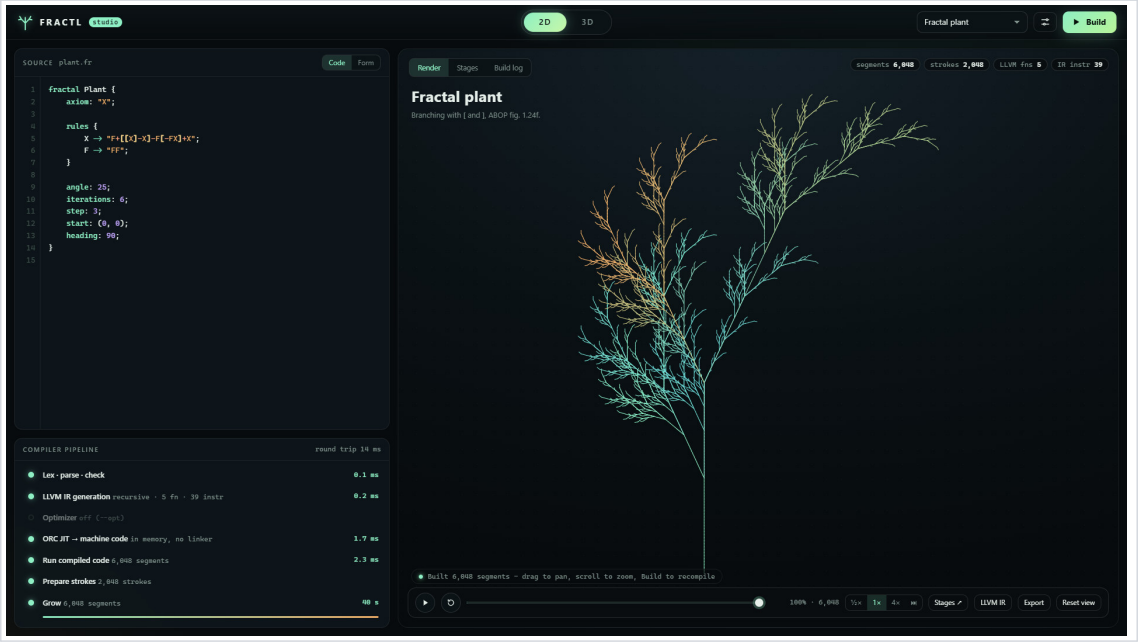


Figure 7. Fractal plant (`plant.fr`): 6,048 segments, 2,048 strokes, 5 LLVM functions, 39 IR instructions

Tree

`X -> F[+X]F[-X]+X` and `F -> FF`, 20 degree turns, 7 iterations: one branch to each side at every node.



Figure 8. Tree (`tree.fr`): 4,118 segments, 1,458 strokes, 5 LLVM functions, 33 IR instructions

Bush

`X -> F[+X][-X]FX` and `F -> FF`, 25.7 degree turns, 7 iterations: two side branches per node, then the stem grows on.

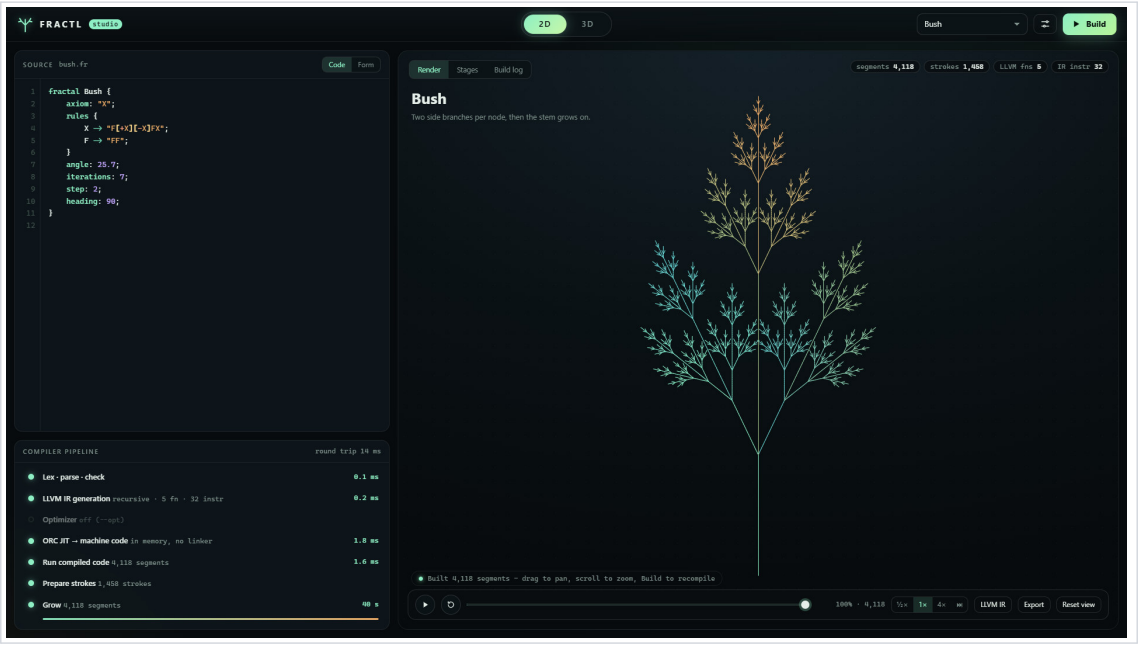


Figure 9. Bush (`bush.fr`): 4,118 segments, 1,458 strokes, 5 LLVM functions, 32 IR instructions

Weed

A single rule, `F -> FF-[-F+F+F]+[+F-F-F]`, 22.5 degree turns, 4 iterations. Every segment sprouts a left and a right twig.

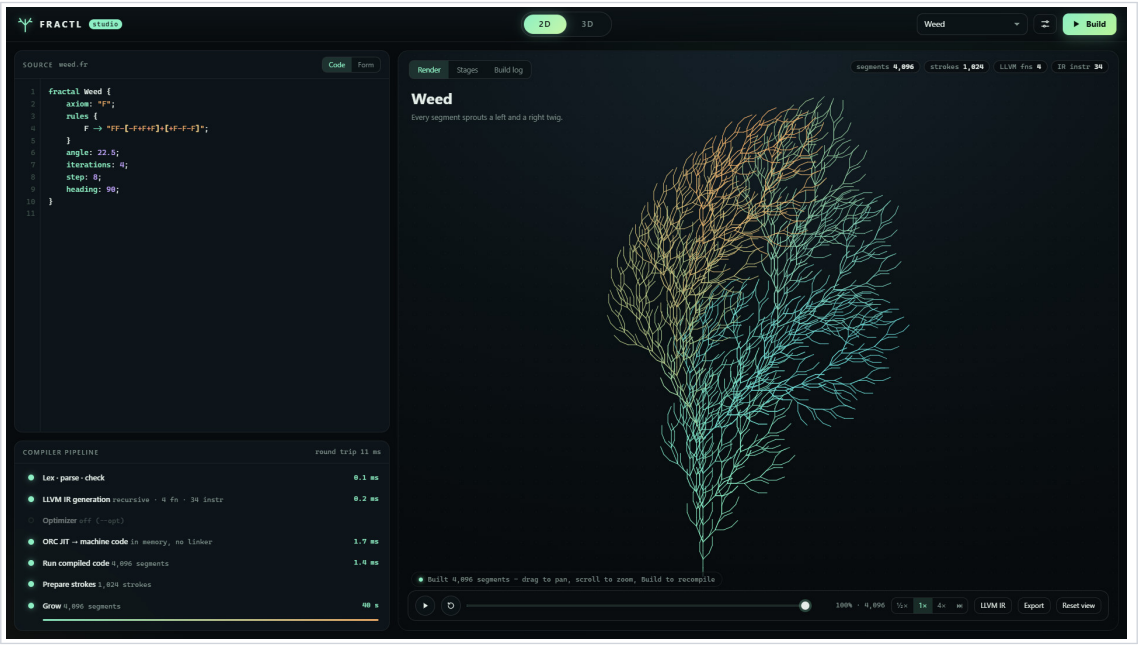


Figure 10. Weed (`weed.fr`): 4,096 segments, 1,024 strokes, 4 LLVM functions, 34 IR instructions

Koch snowflake

Stage 1's demo: axiom `F--F--F`, rule `F -> F+F--F+F`, 60 degree turns, 4 iterations. It is one continuous line of 768 segments.

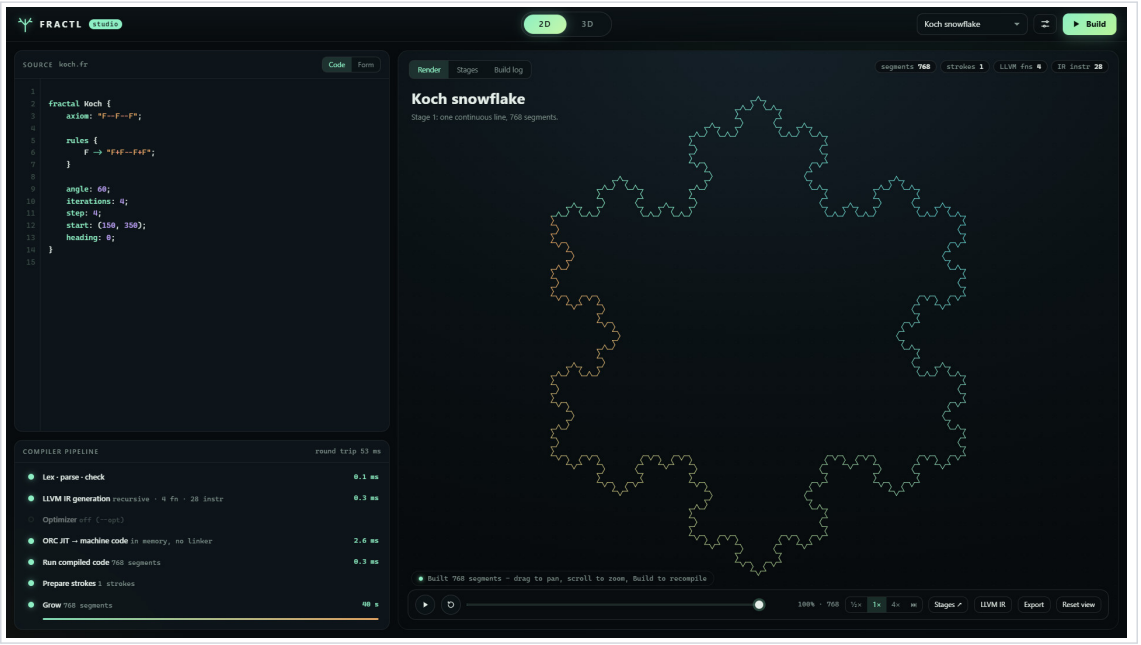


Figure 11. Koch snowflake (`koch.fr`): 768 segments, 1 stroke, 4 LLVM functions, 28 IR instructions

Heighway dragon

Axiom `FX` with `X -> X+YF+` and `Y -> -FX-Y`, 90 degree turns, 12 iterations. `X` and `Y` draw nothing; they only steer the rewriting.



Figure 12. Heighway dragon (`dragon.fr`): 4,096 segments, 1 stroke, 5 LLVM functions, 29 IR instructions

Sierpinski carpet

`F -> F+F-F-F-f+F+F+F-F` and `f -> fff`, 90 degree turns, 4 iterations. Lowercase `f` moves without drawing, which leaves the holes.

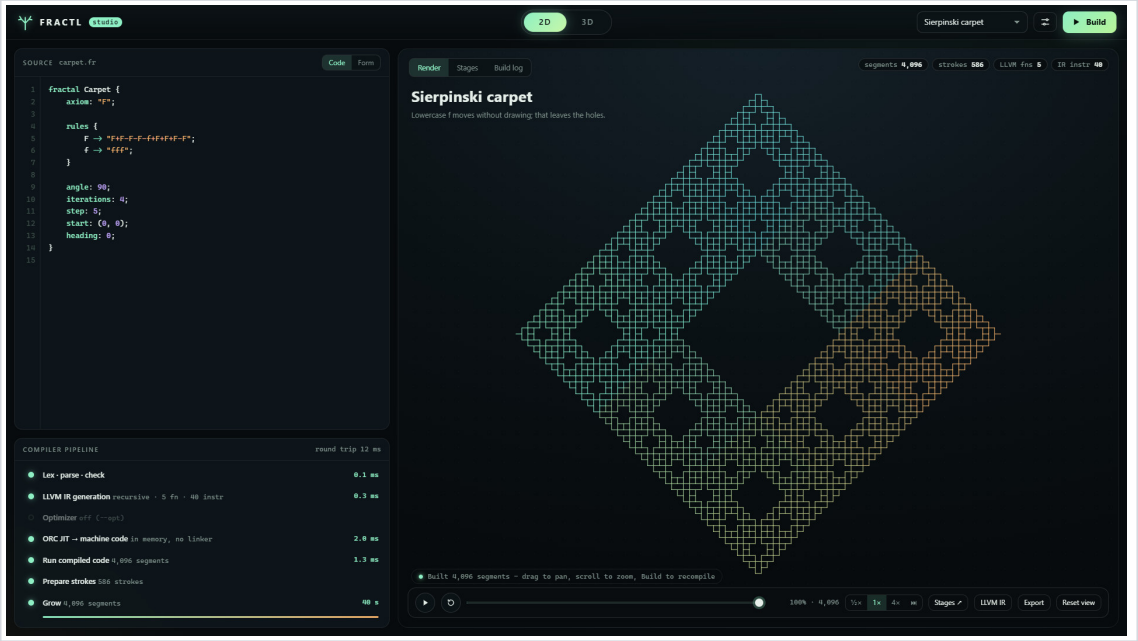


Figure 13. Sierpinski carpet (`carpet.fr`): 4,096 segments, 586 strokes, 5 LLVM functions, 40 IR instructions

Sierpinski arrowhead

`F -> G-F-G` and `G -> F+G+F`, 60 degree turns, 7 iterations. Both `F` and `G` draw, and the alternating turns build the triangle.



Figure 14. Sierpinski arrowhead (`sierpinski.fr`): 2,187 segments, 1 stroke, 5 LLVM functions, 30 IR instructions

Hilbert curve

`A -> -BF+AFA+FB-` and `B -> +AF-BFB-FA+`, 90 degree turns, 6 iterations. It is a space-filling curve; `A` and `B` only steer.



Figure 15. Hilbert curve (`hilbert.fr`): 4,095 segments, 1 stroke, 5 LLVM functions, 40 IR instructions

Gosper curve

`F -> F-G--G+F++FF+G-` and `G -> +F-GG--G-F++F+G`, 60 degree turns, 4 iterations: the hexagonal flowsnake.



Figure 16. Gosper curve (`gosper.fr`): 2,401 segments, 1 stroke, 5 LLVM functions, 50 IR instructions

Triangle

Stage 1's first integration test: axiom `F--F--F` with `iterations: 0`, so the program is just 3 forward moves and 4 turns. It renders point-down because SVG's y-axis points down.



Figure 17. Triangle (`triangle.fr`): 3 segments, 1 stroke, 4 LLVM functions, 21 IR instructions

5.5.3 Compiler stages and the Stage 1 toolchain

These screenshots show the intermediate results of the compiler and the original ahead-of-time toolchain, using the Koch snowflake grammar and the Heighway dragon.

Frontend stages: source, AST, tokens and turtle IR

The compiler stage viewer shows each intermediate result of the frontend for `koch.fr`, side by side.

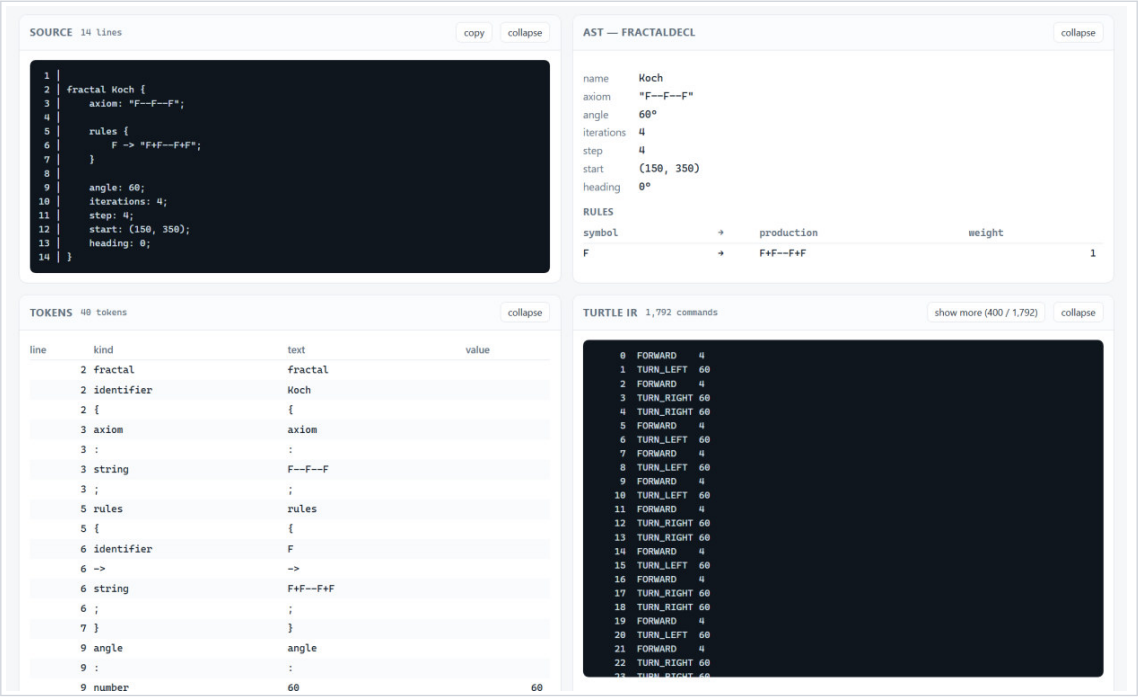


Figure 18. Frontend stages: source, AST, tokens and turtle IR

- **Source:** the 14-line `fractal Koch { ... }` declaration.
- **Tokens:** the lexer produces 40 tokens (keywords, identifiers, strings, numbers, punctuation), each with its line number.
- **AST:** the parser builds a FractalDecl node with the axiom, the angle, the iterations, the step, the start, the heading and the rule table (F -> F+F--F+F).
- **Turtle IR:** 4 rewrite passes expand the 7-symbol axiom to 1,792 symbols. These are lowered to 1,792 commands: FORWARD, TURN_LEFT and TURN_RIGHT.

Generated LLVM IR (koch.ll)

The backend turns the turtle IR into a real LLVM module for the x86_64-pc-linux-gnu target. This is the Stage 1 unrolled form; for `koch.fr` it has 1,826 lines.



Figure 19. Generated LLVM IR (koch.ll)

- The runtime interface is declared as external functions: `@turtle_init`, `@turtle_forward`, `@turtle_turn`, `@turtle_finish`, and others.
- The output SVG path is stored in the module as a private string constant (`@svg_path`), so the compiled program itself writes the image.
- All parameters (angle, step, start position) are known at compile time and folded into double constants in every call, for example `call void @turtle_init(double 4.0e+00, double 6.0e+01, ...)`.
- The viewer can also show the -O0 and optimized IR, one at a time or side by side.

End-to-end build and run from Windows PowerShell

A real PowerShell session that compiles `koch.fr` to a native object, links it with the runtime, and runs the program. The Linux toolchain steps (`fractlc`, `gcc` and the compiled program) are called through `wsl`. The frontend runs as a native Windows build (`fr_frontend.exe`).

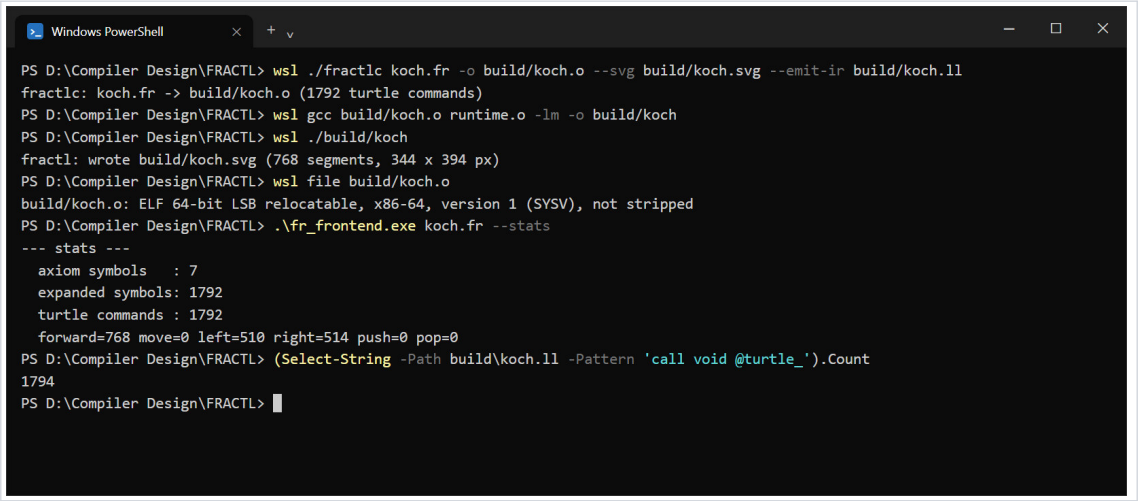


Figure 20. End-to-end build and run from Windows PowerShell

- `fractlc` reads `koch.fr` and emits `koch.o` (1,792 turtle commands). It also writes the IR to `koch.ll`.
- `file` confirms that the output is a genuine ELF 64-bit x86-64 relocatable object, produced by the LLVM TargetMachine.
- Linking with `runtime.o` using `gcc` gives an executable. Running it writes `koch.svg` with 768 segments (344 x 394 px).
- `fr_frontend.exe --stats` reports the command counts: forward = 768, left = 510, right = 514. The IR contains 1,794 turtle calls (1,792 commands plus init and finish).

Rendered output in the stage viewer

The SVG that the compiled executable writes, shown in the stage viewer with zoom and pan. The runtime tracks the bounding box of every point and sizes the SVG to fit, so the image is never clipped.

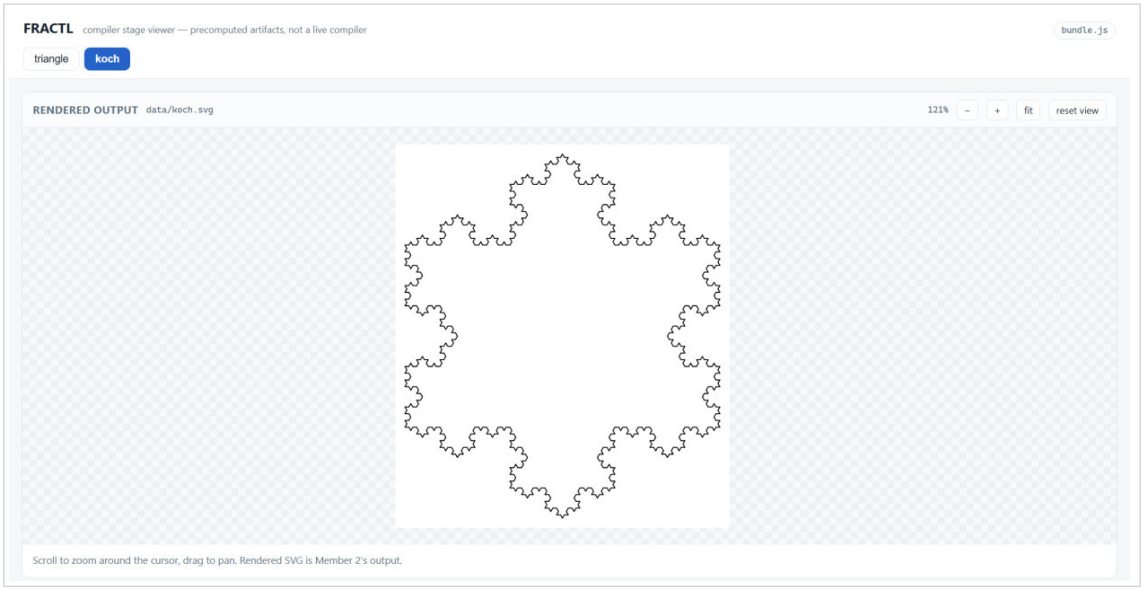


Figure 21. Rendered output: Koch snowflake

Interactive web front end: Heighway dragon

`web_ui.py` is the Stage 1 browser form in front of the real compiler. The server writes a genuine `.fr` file, runs `fractlc`, links the object with `cc`, and runs the binary. For the dragon it reported 12,286 turtle commands, 4,096 segments and a 495 x 335 px picture.

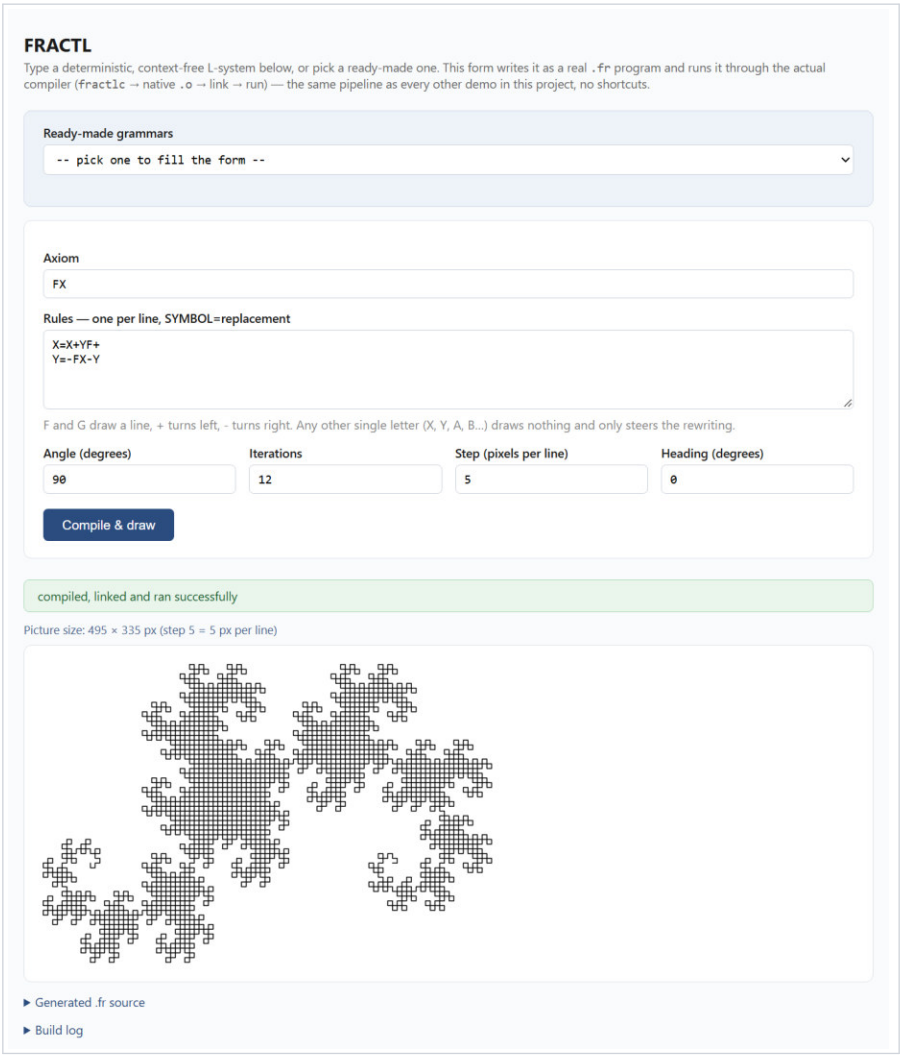


Figure 22. Interactive web front end: Heighway dragon

6. Conclusion

FRACTL shows that a compact DSL can go through every classic compiler phase and still produce rich results: scanning, LL(1) parsing, semantic checks, grammar expansion, an intermediate representation, optimization, LLVM code generation, and both ahead-of-time and JIT execution.

The key design result is recursive code generation. It keeps the generated IR the size of the grammar (531 lines against 201,463 for the meadow scene) while producing byte-identical drawings. Stochastic rules with a shared random generator keep the output reproducible, and the automated test suite (79,580 frontend checks, plus golden files for every demo) keeps earlier stages from regressing.

Known limits: parametric L-systems (`F(x)`) and context-sensitive rules are not supported, leaves are blades rather than arbitrary polygons, and 3D drawings beyond a few hundred thousand segments make the browser slow to upload. Natural extensions are parametric rules, richer leaf geometry and

more optimization passes.

7. Project Repository Link

(To be added.)